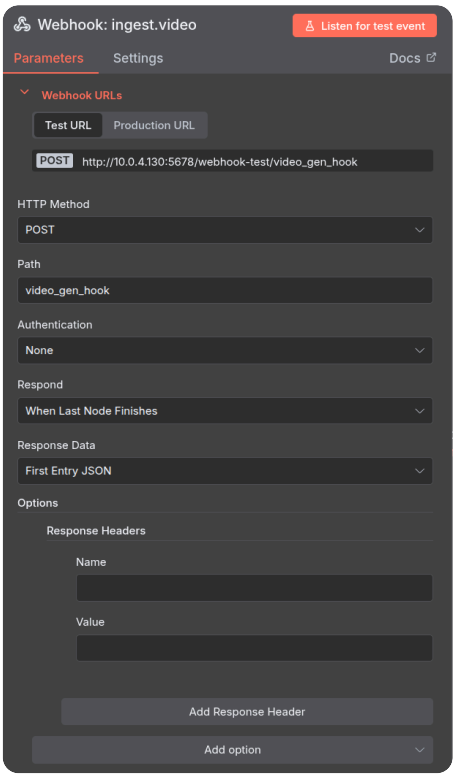
Webhook trigger configuration



Consider the project document titled 'Video Gen & Agent 1 Webhook.'
Let's work through the following section and configure the Webhook trigger:
Webhook Trigger Configuration
Node: Webhook: ingest.video
Path: video_gen_hook
Method: POST
Response Mode: onReceived (returns 202 immediately)
Authentication: Requires x-ingest-key header matching $env.INGEST_TOKEN

Consider the node parameters available in the attached screenshot.

Focus on the Respond section. The options in the drop-down are as follows:
When Last Node Finishes,

Consider the project document titled 'Video Gen & Agent 1 Webhook.'
Let's work through the following section and configure the Webhook trigger: Webhook Trigger Configuration
Node: Webhook: ingest.video
Path: video_gen_hook
Method: POST
Response Mode: onReceived (returns 202 immediately)
Authentication: Requires x-ingest-key header matching $env.INGEST_TOKEN

____

Consider the node parameters available in the attached screenshot.
Focus on the 'Respond' section and the 'Response Data' section.
The options in the Respond drop-down list are as follows:
When Last Node Finishes,
Immediately,
Using 'Respond to Webhook' Node,
Streaming

The options in the Response Data drop-down list are as follows:
All entries,
FirstEntryJSON,
FirstEntryBinary,
NoResponseBody

you wrote the following:
Response Mode: onReceived (returns 202 immediately)

The workflow expects a POST payload with:

```
json
{
  "source_url": "https://example.com/video.mp4",
  "label": "happy" (optional),
  "meta": { "generator": "luma" }
}
```

Remember, I am a new N8N user, so I don't understand the parameters. Explain these options in detail.
Let me know how to set the parameters based on the provided information.

Thought for 1m 13s >

Cool, this is a good place to slow down and really grok what n8n is asking you for.

You've already got the *what* from the doc:

- Webhook path: `video_gen_hook`
- Method: `POST`
- Auth: `x-ingest-key` must match `$env.INGEST_TOKEN`
- Response mode: **"onReceived (returns 202 immediately)"**

Now we just need to map that to the UI options you see.

---

1. The Respond options (response *timing*)

Think of this as: *"When should n8n send an HTTP response back to the caller?"*

1. **When Last Node Finishes**
   - n8n waits until the whole workflow is done (Media Mover call, polling, DB insert, etc.).
   - The HTTP request stays open the whole time.
   - Good for quick synchronous things.
   - **Bad for you here**, because ingest can easily take several seconds and you *don't* want the web app blocked.
2. **Immediately**
   - As soon as the Webhook node fires and n8n has the request, it responds.
   - The rest of the workflow keeps running in the background.
   - Typical pattern for "accept job, process async".
   - This is exactly what the doc means by **"Response Mode: onReceived (returns 202 immediately)"**. LUMA_N8N_INTEGRATION_GUIDE
3. **Using "Respond to Webhook" Node**
   - You place a separate *Respond to Webhook* node somewhere later in the workflow.
   - The response is sent **when that node runs**, not when the Webhook triggers.
   - Useful for more complex branching ("respond early on error, later on success"), but overkill for Agent 1 right now.
4. **Streaming**

- Keeps the connection open and streams chunks of data (Server-Sent Events style).
- More for chatty, real-time UIs.
- Not what you want for a simple “ingest this video” hook.

👉 **For Agent 1 you want:** `Respond = Immediately`**.**
That’s n8n’s UI label for the `responseMode: "onReceived"` setting used in the docs.

---

2. The Response Data options (response *content*)

This is: *“What should the HTTP response body contain?”* n8n thinks in terms of “items” flowing through the workflow; each item is basically one JSON object.

The options:

1. **All entries**
   - Returns an array of all items currently in the workflow at response time.
   - For simple webhooks, this usually means an array with one element (the request body).
   - Kind of noisy for our use.
2. **First Entry JSON**
   - Returns just the first item as a JSON object.
   - For a Webhook trigger, that first item is usually the parsed request body.
   - So the caller basically gets back what they sent (or whatever you’ve changed it to by that point).
3. **First Entry Binary**
   - Returns the binary data of the first item (files, images, etc.).
   - Used when your webhook is effectively a download endpoint.
   - Not what you need for ingest.
4. **No Response Body**
   - Sends only the HTTP status + headers; the body is empty.
   - The caller just sees `202 Accepted` and nothing else.

For your use case:

- The caller (your web app / Luma client) mostly just needs to know:
  - “Did n8n accept this job?” → status code `202`

- Maybe some small JSON like `{ "status": "accepted" }` (nice, but not essential).

The docs and summary guides talk about a `202` with a small JSON ack (sometimes including a `correlation_id`).

So you've got two sane choices:

Option A (simple + matches most of the docs)

- **Respond**: `Immediately`
- **Response Data**: `First Entry JSON`
- Under **Options**:
  - Set **Response Code** to `202`.

Result:

- The caller gets HTTP `202`.
- The body will be the JSON of the first item (usually the same payload it sent).
- This is fine for the web app; it already knows what it sent.

Option B (very strict "just a 202")

- **Respond**: `Immediately`
- **Response Data**: `No Response Body`
- Under **Options**:
  - Set **Response Code** to `202`.

Result:

- The caller just sees `202 Accepted` with an empty body.
- Also totally valid for an async ingest API.

If you want to stay closest to the text in `LUMA_N8N_INTEGRATION_GUIDE.md` and `Video Gen & Agent 1 Webhook` but keep it simple, I'd do:

> ***Set*** *`Respond = Immediately`* ***and*** *`Response Data = First Entry JSON`****, with Response Code = 202.***

That gives you:

- Async behavior (web app doesn't hang).
- A clear "I heard you" response.

- Correct status code.

---

3. How this fits your payload

Your web app (or curl) sends:

```
{
  "source_url": "/media/.../videos/temp/test.mp4",
  "label": "happy",
  "meta": { "generator": "luma" }
}
```

With:

```
POST /webhook/video_gen_hook
x-ingest-key: <INGEST_TOKEN>
Content-Type: application/json
```

- n8n Webhook node checks auth (downstream node).
- Because **Respond = Immediately**, n8n sends back the 202 as soon as the Webhook node fires.
- The rest of the workflow (Media Mover call, polling, DB insert, emit event) runs in the background as Agent 1's actual work.

If you want, next step we can click through the actual Webhook node UI together and map each sub-field ("Response Code", "Response Headers", etc.) to the spec so nothing feels mysterious.

yes. Let's click through the actual Webhook node UI together

Perfect. Let’s go step-by-step as if you’re sitting in front of the n8n editor.

🧭 1. Create or open the workflow

- Open **n8n** (your local instance).
- Either open your existing *Reachy_Local_08.4.2 ingest workflow* or create a new one.
- Add a node and choose **Webhook** as the type.
- Rename it to `Webhook: ingest.video` (that’s just a label, but helpful later).

⚙️ 2. Basic configuration (top section)

Field	What to enter	Why
HTTP Method	`POST`	Matches the spec: web app or Luma agent will `POST` video info
Path	`video_gen_hook`	Creates an endpoint like `http://<n8n_host>:5678/webhook/video_gen_hook`
Response Mode (Respond)	`Immediately`	This is what “onReceived (returns 202 immediately)” means
Response Data	`First Entry JSON`	n8n returns the first JSON object (the payload you sent)
Response Code	`202`	Tells the caller “Accepted — processing in background”

✅ That’s the main behavior done.

🔐 3. Authentication setup (lower in the Webhook node)

Scroll down to the **Authentication** section. Choose **Header Auth** (if visible) or configure via **Options → Headers** if your UI version doesn’t have a dedicated field.

- Under **Header Name**, type: `x-ingest-key`

- Under **Expected Value**, enter: `{{$env.INGEST_TOKEN}}`

This means:

> *"Only requests that include `x-ingest-key: <your token>` will be accepted."*

If your n8n version doesn't expose `$env` directly here:

- Leave Expected Value blank for now,
- Add a small **IF** node after the Webhook node to check `{{$json["headers"]["x-ingest-key"]}} === $env.INGEST_TOKEN`.

🧩 4. Respond section (click the Respond dropdown)

You'll see:

- **When Last Node Finishes**
- **Immediately**
- **Using 'Respond to Webhook' Node**
- **Streaming**

👉 Select **Immediately**

Below that:

- **Response Data**: select **First Entry JSON**
- **Response Code**: set to `202`
- **Response Headers (optional)**:
  Add one if you like:

Key	Value
`Content-Type`	`application/json`

💡 5. Test the endpoint

Once you hit **Execute Node** or **Activate Workflow**, n8n will show you a URL like:

```
http://localhost:5678/webhook/video_gen_hook
```

If you're running behind a reverse proxy, it'll use that address.

Now test it from your terminal:

```
curl -X POST http://localhost:5678/webhook/video_gen_hook \
  -H "Content-Type: application/json" \
  -H "x-ingest-key: your_token_here" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

You should see:

```
HTTP/1.1 202 Accepted
Content-Type: application/json

{"source_url":"https://example.com/video.mp4","label":"happy","meta":
{"generator":"luma"}}
```

If you used “No Response Body,” you’d just see the `202 Accepted` line with no JSON.

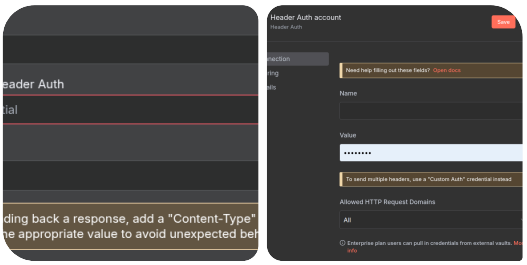
🧠 6. Next steps in the workflow

After this Webhook node:

- You’ll likely connect to an **HTTP Request** node or **Function node** that:
  - Validates the payload
  - Calls Media Mover API to persist it
  - Emits a message to Agent 1 (e.g., via another webhook or DB insert)

But for now, you’ve got the trigger fully configured.

Do you want me to walk you through setting up the **authorization check** (verifying `x-ingest-key` against `$env.INGEST_TOKEN`) using a lightweight Function or IF node next?



you wrote:
Authentication setup (lower in the Webhook node)

Scroll down to the Authentication section. Choose Header Auth (if visible) or configure via Options → Headers if your UI version doesn't have a dedicated field.

Under Header Name, type: x-ingest-key

Under Expected Value, enter: {{$env.INGEST_TOKEN}}

This means:

"Only requests that include x-ingest-key: <your token> will be accepted."

If your n8n version doesn't expose $env directly here:

Leave Expected Value blank for now,

Add a small IF node after the Webhook node to check {{$json["headers"]["x-ingest-key"]}} === $env.INGEST_TOKEN.

When I select the Header Auth option under Authentication, I am prompted to create a credential for Header Auth.
The Header Auth Account window is provided to show you the options.

Good — you're in the right spot. n8n is asking you to define what header name and value it should expect when authenticating webhook calls.
Here's exactly how to fill that "Header Auth Account" dialog so it matches your project spec.

🧩 Step-by-step setup for `x-ingest-key` authentication

1. In the **Webhook node**, set **Authentication** = `Header Auth`.
   You'll see the **Credential for Header Auth** dropdown turn red until a credential exists.
2. Click **"Add New" → "Header Auth Account."**
   That opens the window you just showed in your screenshot.
3. Fill the fields as follows:

Field	Value	Explanation
Name	`x-ingest-key`	This tells n8n *which* header to check for authentication.
Value	`{{$env.INGEST_TOKEN}}`	Pulls your secret key from n8n's environment variable. (You can paste the actual token here if you prefer, but using `$env` keeps it secure and consistent.)
Allowed HTTP Request Domains	`All`	Accepts requests from any domain; fine for a local system.

4. Click **Save**.
5. Back in the Webhook node:
   - Under **Credential for Header Auth**, pick the credential you just made.
   - You'll now see the red warning disappear.

🧠 What this does

When a POST request hits your webhook, n8n checks for:

```
x-ingest-key: <token>
```

If the value matches `$env.INGEST_TOKEN` from your environment file, the workflow runs.
If it’s missing or wrong, n8n immediately returns a **401 Unauthorized**.

⚙️ Optional: verify `$env.INGEST_TOKEN`

If you’re not sure n8n can see that environment variable:

- Check your `.env` or systemd service for `INGEST_TOKEN=<your_token>`
- Restart n8n afterward.

You can also temporarily type the raw token into **Value** for testing, then switch to `{{$env.INGEST_TOKEN}}` later.

Want to go ahead and add the `"Content-Type": "application/json"` header next, so the warning about “unexpected behavior” disappears?

you wrote:
If you’re not sure n8n can see that environment variable:

Check your .env or systemd service for INGEST_TOKEN=<your_token>

_____
Note the .env file reads as follows:
N8N_INGEST_TOKEN=tkn3848

It is not 'INGEST_TOKEN.'
Or simply use the following for the value:
{{$env.N8N_INGEST_TOKEN}}

Yes. Add the "Content-Type": "application/json" header next

Perfect catch — exactly right.
Your environment variable is named `N8N_INGEST_TOKEN`, so the correct expression inside n8n should be:

```
{{$env.N8N_INGEST_TOKEN}}
```

That's the value you'll use in your **Header Auth credential** under **Value**.
Here's the updated setup, step by step — plus how to add that Content-Type header:

✅ Final Webhook: ingest.video configuration

Authentication

Field	Setting	Notes
Authentication	**Header Auth**	Enforces the header check
Credential for Header Auth	*(select the one you just created)*	Points to your `x-ingest-key` credential
Name	`x-ingest-key`	The header key expected in the incoming request
Value	`{{$env.N8N_INGEST_TOKEN}}`	Matches your `.env` file variable
Allowed HTTP Request Domains	`All`	Local workflow, no restriction needed

This ensures that a request must include:

```
x-ingest-key: tkn3848
```

or whatever the `.env` variable resolves to.

📦 Respond section

Field	Setting	Notes
Respond	**Immediately**	Returns HTTP 202 instantly ("onReceived")
Response Data	**First Entry JSON**	Echoes back the payload as JSON

Field	Setting	Notes
Response Code	**202**	Signals “Accepted”

🧾 Add “Content-Type” header

Scroll to the **Options → Response Headers** section and add:

Name	Value
`Content-Type`	`application/json`

That clears the yellow warning and ensures clients see the response body properly formatted.

🧪 Quick test

Run this from your terminal (replace the token if needed):

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "x-ingest-key: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

Expected response:

```
HTTP/1.1 202 Accepted
Content-Type: application/json

{"source_url":"https://example.com/video.mp4","label":"happy","meta":
{"generator":"luma"}}
```

Would you like to wire this webhook into the next node (the one that forwards the payload to the Media Mover API) next?

I tried the following command:

```
curl -X POST
http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "x-ingest-key: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

_____

The workflow is active. I disconnected the webhook node from the rest of the workflow to simplify the test. When I click the Execute Workflow button, I receive the following message:

Stopped thinking >

```
curl -X POST
http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "x-ingest-key: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

_____

The workflow is active. I disconnected the webhook node from the rest of the workflow to simplify the test. When I click the Execute Workflow button, I receive the following message while waiting for trigger event: Waiting for you to call the test URL (displayed over the Webhook node).

When I double-click to open the webhook node, I see that it's telling me to 'Make a POST request to: http://10.0.4.130:5678/webhook-test/video_gen_hook

Simply pasting the URL into a browser returns the following message:
{"code":404,"message":"This webhook is not registered for GET requests. Did you mean to make a POST request?"}

What am I missing in the process?

Nice, you're super close. You just ran into one of n8n's more confusing "features": **test URL vs production URL** *and* **GET vs POST**.

Let's untangle it.

---

1. Two different URLs for the same Webhook node

In the Webhook node you'll see:

- **Test URL →** `http://10.0.4.130:5678/webhook-test/video_gen_hook`
- **Production URL →** `http://10.0.4.130:5678/webhook/video_gen_hook`

These are **not** the same thing.

When you click Execute Workflow

- n8n switches the webhook into **manual test mode**.
- It shows:

  > *"Waiting for you to call the test URL..."*

- In that mode, only **the Test URL** will trigger the run:

  ```
  http://10.0.4.130:5678/webhook-test/video_gen_hook
  ```

But your curl command is hitting:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook ...
                        ^^^^^^^
                  production URL
```

So n8n is sitting there waiting for `/webhook-test/...`, and your POST is going to `/webhook/...`.
Result: the manual execution never starts, so the UI stays on "Waiting for trigger event."

---

2. Why the browser shows a 404

You pasted the URL in the browser and saw:

> `{"code":404,"message":"This webhook is not registered for GET requests. Did you mean to make a POST request?"}`

That's expected:

- Browsers send **GET** when you just paste a URL.
- Your webhook is configured for **POST** only.
- So n8n correctly says: "I don't have a GET handler for this."

So:

- Browser → fine for a quick "is server up?" check.
- But for the real trigger, you must use **POST** (curl / Postman / your app).

---

3. How to test correctly (two clean options)

Option A – Test mode (using Execute Workflow)

1. **Deactivate** the workflow toggle (to avoid confusion with prod URL).
2. Click **Execute Workflow**.
3. n8n now shows the **Test URL** on the Webhook node:

   ```
   http://10.0.4.130:5678/webhook-test/video_gen_hook
   ```

4. In a terminal, run **this** (note `webhook-test`):

   ```
   curl -X POST http://10.0.4.130:5678/webhook-test/video_gen_hook \
     -H "x-ingest-key: tkn3848" \
     -H "Content-Type: application/json" \
     -d '{
           "source_url": "https://example.com/video.mp4",
           "label": "happy",
           "meta": {"generator": "luma"}
         }'
   ```

5. You should see the execution complete in the n8n UI, because the test run caught that webhook.

Option B – Production mode (no Execute Workflow)

1. **Activate** the workflow (toggle ON).

2. **Do not** click Execute Workflow now.
3. From your terminal, call the **Production URL**:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "x-ingest-key: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

4. Because we set:
   - **Respond:** Immediately
   - **Response Code:** 202
   - **Response Data:** First Entry JSON
     you should get back a `202 Accepted` with the JSON body.

TL;DR what you were “missing”

- **Execute Workflow** expects `/webhook-test/...`, not `/webhook/...`.
- Pasting the URL into a browser sends **GET**, but your webhook is **POST-only**.

Once you line those two things up, you’re good.

If you tell me which mode you want to use long-term (manual test vs prod), I can walk you through what you should see in the n8n “Execution” list so you can double-check everything’s firing the way Agent 1 expects.

I'm still receiving the following error, whether I use the test URL or the production URL:
{"code":0,"message":"There was a problem executing the workflow"}

_____

Should I be running the curl command from a particular terminal?

Good question — that error points to n8n itself catching the webhook but then failing internally, before or during execution.
Let's unpack what's happening and how to isolate it.

🧠 First: where the curl should run

You can run it from **any terminal** that can reach your n8n instance.

- If you're testing locally on the same Ubuntu host where n8n is running → you can run it right there.
- If you're testing from another machine on the LAN → ensure the IP `10.0.4.130` is reachable and the port `5678` isn't firewalled.

So no, there's no "special" terminal requirement — this message isn't about where the curl runs.
It means n8n received your request but hit an execution error inside the workflow.

🧩 Step-by-step to locate the issue

1. Check the Executions list

In n8n's left sidebar, click:

```
Executions → View past executions
```

You should see your failed webhook attempts listed there.

Click the most recent one. The top of the detail view usually shows something like:

```
Error: There was a problem executing the workflow
```

Expand the "stack" or "details" section — you'll likely see a node name and the precise error message.
This tells us *which node* failed.

If the workflow is just a single **Webhook node**, this points to the node's configuration itself (auth mismatch, malformed env, etc.).

---

2. Simplify: test without auth first

Temporarily set **Authentication → None** on the Webhook node.
Save and reactivate.

Then run:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Content-Type: application/json" \
  -d '{"test": "ok"}'
```

If that works (returns `202` and you see a success in Executions), the error was in the **Header Auth** credential.

If it still fails, the error is in the Webhook node's configuration itself (like an invalid response code or an expression issue).

---

3. If auth is the issue

Re-enable **Header Auth** and double-check:

- The credential's **Name** = `x-ingest-key`
- The **Value** = `{{$env.N8N_INGEST_TOKEN}}`
- Confirm the env var actually exists. From your host shell (where n8n runs):

  ```
  echo $N8N_INGEST_TOKEN
  ```

  If it's blank, n8n will throw an internal error when evaluating that expression.

If your `.env` uses `N8N_INGEST_TOKEN=tkn3848`, make sure that file is loaded when n8n starts.
If you're running via `systemctl`, check the service file's `EnvironmentFile=` line.

---

4. Add basic error-proofing

If you keep getting a blank 500, add a quick **Set** node after the Webhook with something simple like:

```
status: "received"
```

and respond with that. This can help confirm that the Webhook node itself executes fine.

---

5. Quick test matrix

Mode	Auth	Expected Result
Test URL, Auth=none	Success	proves connectivity
Production URL, Auth=none	Success	proves workflow logic
Production URL, Auth=HeaderAuth	If fails → env token issue	isolate environment error

---

If you can paste (or screenshot) the **Execution Details** section of the failed run — the part that shows the red error message and node name — I can pinpoint exactly what’s tripping n8n.

Remember, I am using the self-hosted version of N8N. I don't see 'Executions' in the left sidebar.

Not sure if I can enable execution history

Right — that tracks.
In the **Community Edition (self-hosted)**, execution data isn't always stored or shown by default — especially if you're running in "production mode" (`N8N_EXECUTIONS_PROCESS=main` or `N8N_SKIP_WEBHOOK_DEREGISTRATION=true`).

Let's walk through what's going on and how to see what n8n actually did with your request.

🧭 1. Confirm where n8n logs its errors

Since you're running n8n locally (probably as a systemd service or from a terminal), you can watch logs directly.

If you started n8n manually (for example):

```
n8n start
```

then you'll see live errors printed right in that same terminal when you hit the webhook.

If you're running n8n as a **systemd service**, check:

```
sudo journalctl -u n8n -f
```

then send your `curl` request again.
If there's an evaluation problem (like `$env.N8N_INGEST_TOKEN` not being set or a malformed JSON), you'll see it appear there immediately.

🧩 2. Temporarily enable saved executions

You can turn on execution logging so runs appear in the UI — even on self-hosted.

Add these lines to your environment (either `.env` or systemd unit file):

```
N8N_LOG_LEVEL=debug
N8N_EXECUTIONS_MODE=regular
N8N_EXECUTIONS_DATA_SAVE_ON_ERROR=all
N8N_EXECUTIONS_DATA_SAVE_ON_SUCCESS=all
```

Then restart the service:

```
sudo systemctl restart n8n
```

Now you should see a **“Executions”** menu appear in the sidebar.
It sometimes hides behind the “...” menu at the bottom in newer builds.

🧪 3. Minimal sanity check

Try the simplest possible workflow:

1. Create a new workflow.
2. Add a **Webhook** node.
   - Path: `test`
   - Method: `POST`
   - Respond: `Immediately`
   - Response Data: `First Entry JSON`
   - Authentication: `None`
3. Activate the workflow.

Then run:

```
curl -X POST http://10.0.4.130:5678/webhook/test \
  -H "Content-Type: application/json" \
  -d '{"msg": "hello"}'
```

Expected output:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"msg":"hello"}
```

If this one fails the same way (`{"code":0,"message":"There was a problem executing the workflow"}`), the issue is global — likely a permissions or environment variable evaluation error.

🧠 4. If it works without auth

If the minimal version works fine, re-enable **Header Auth** and plug your `$env.N8N_INGEST_TOKEN` expression back in.
Then restart n8n again so the environment variable is picked up.

You can verify n8n actually sees that variable from within the Webhook node:

- In the Value field, click **“Execute Expression”** (little gear icon).
- It should display the token (`tkn3848`).
  If it shows *empty*, n8n can’t see it — that’s why it’s crashing.

Want to check your service configuration next (so we can confirm that the `.env` is being loaded properly)?
If you show me how n8n is launched on Ubuntu — for instance, via `sudo systemctl cat n8n` or a custom shell script — I can point out exactly where to declare the env variable.

I'm running an 8n in a docker container. How does that affect the testing and other functionality?

Remember in the current workflow, I disconnected the webhook node from the rest of the workflow.

Thought for 27s >

Yeah, Docker changes the rules a bit here — not for *how* the webhook works, but for *where* config lives and how env vars actually get into n8n.

Let's untangle this in two passes:

1. What Docker changes in general
2. Concrete steps to get your webhook test working

---

1. What Docker changes (for n8n + your webhook)

Running n8n in Docker means:

- **All environment variables must be passed into the container**, not just set on the host.
  - `$N8N_INGEST_TOKEN` only exists *inside* n8n if you pass it in via `docker run -e ...` or `docker-compose.yml → environment:`.
  - If it's only in a host `.env` file and never wired into the container, then `{{$env.N8N_INGEST_TOKEN}}` inside n8n evaluates to **empty**, which can crash the node.
- **Ports are mapped**:
  - You're hitting `10.0.4.130:5678` — that's the *host* port mapped to the container's internal port.
  - That's fine as long as your `docker run` (or compose) has something like `-p 5678:5678`.
- **Executions UI & logging** are driven by env vars *inside the container*, not on the host:
  - `N8N_EXECUTIONS_MODE`, `N8N_EXECUTIONS_DATA_SAVE_ON_ERROR`, etc. all must be set in the container's environment.

So the short version: everything we've been talking about (`N8N_INGEST_TOKEN`, execution mode, log level) has to be configured in **Docker**, not just in your normal shell.

---

2. Let's fix your webhook test step-by-step

Right now, you've:

- Disconnected the Webhook node from the rest of the workflow ✅
- Set **Header Auth** using `{{$env.N8N_INGEST_TOKEN}}` ✅
- curl returns:
  `{"code":0,"message":"There was a problem executing the workflow"}` ❌

That strongly suggests the node is blowing up on auth / env evaluation.

We'll do this in three quick steps.

Step 1 – Prove the webhook works *without* auth

Inside the n8n UI:

1. Open your `Webhook: ingest.video` node.
2. Set **Authentication** = `None`.
3. Keep:
   - **Respond**: `Immediately`
   - **Response Data**: `First Entry JSON`
   - **Response Code**: `202`
4. **Activate** the workflow.
5. From any terminal on the host, run:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

If that returns `202` and echoes JSON, good — the webhook node itself is fine. The problem is 100% in auth / env.

If this *still* fails with `code:0`, we'll check container logs in a sec.

Step 2 – Make sure `N8N_INGEST_TOKEN` actually exists inside the container

From the Ubuntu host, find the container:

```
docker ps | grep n8n
```

Grab the container name or ID (e.g., `n8n` or `n8n-1`).

Now check env inside the container:

```
docker exec -it <container_name> env | grep N8N_INGEST_TOKEN
```

You want to see:

```
N8N_INGEST_TOKEN=tkn3848
```

If you see nothing, that's the issue.

Fix it (depending on how you run n8n):

If you use `docker run` manually:

Stop the container:

```
docker stop <container_name>
docker rm <container_name>
```

Start it again with the env var:

```
docker run -it --name n8n \
  -p 5678:5678 \
  -e N8N_INGEST_TOKEN=tkn3848 \
  ...other -e and -v options... \
  n8nio/n8n
```

If you use `docker-compose`:

In your `docker-compose.yml`:

```
services:
  n8n:
    image: n8nio/n8n
    ports:
      - "5678:5678"
    environment:
```

```
      - N8N_INGEST_TOKEN=tkn3848
      # plus your other N8N_... envs
```

Then:

```
docker compose down
docker compose up -d
```

Re-check:

```
docker exec -it <container_name> env | grep N8N_INGEST_TOKEN
```

Once that shows up correctly, *then* n8n can safely use `{{$env.N8N_INGEST_TOKEN}}`.

Step 3 – Re-enable Header Auth cleanly

Back in n8n, on your Webhook node:

1. Set **Authentication** = `Header Auth`.
2. In **Credential for Header Auth**, use or create:
   - **Name**: `x-ingest-key`
   - **Value**: `{{$env.N8N_INGEST_TOKEN}}`
   - Allowed domains: `All`
3. Save the node.
4. Activate the workflow again.

Then test from the host:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "x-ingest-key: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

You should now get:

- `HTTP/1.1 202 Accepted`

- JSON body (your payload) or empty if you switched to `No Response Body`.

Bonus: where to see errors in Docker mode

If something *still* blows up:

```
docker logs <container_name> --tail=100 -f
```

Then call the webhook again and watch for a stack trace.

So the Docker answer in one line:

> *Testing is the same (same curl, same URLs), but env vars and n8n config must be wired into the container; otherwise expressions like `{{$env.N8N_INGEST_TOKEN}}` cause exactly the "problem executing the workflow" error you're seeing.*

If you want, you can paste your `docker run` or `docker-compose.yml` (redact secrets), and I'll mark exactly where to drop the env for Reachy's ingest token.

Keep in mind also that I placed the workflow in a folder. Would this change the path/endpoint required in the URL?

Thought for 11s >

Short answer: nope — the folder does **not** change the webhook URL at all.

In n8n:

- **Folders** are just for organization in the UI.
- **Webhook URL** is built from:
  - Your n8n base URL (e.g. `http://10.0.4.130:5678`)
  - The webhook *type*:
    - `/webhook-test/` when you're in **Execute Workflow** (test mode)
    - `/webhook/` when the workflow is **activated** (prod mode)
  - The **Path** field on the Webhook node (e.g. `video_gen_hook`)

So as long as your Webhook node says:

- **Path**: `video_gen_hook`
- **Method**: `POST`

Then your endpoints are:

- Test mode (when you click *Execute Workflow*): `http://10.0.4.130:5678/webhook-test/video_gen_hook`
- Active / production mode (workflow toggle ON, not manually executed): `http://10.0.4.130:5678/webhook/video_gen_hook`

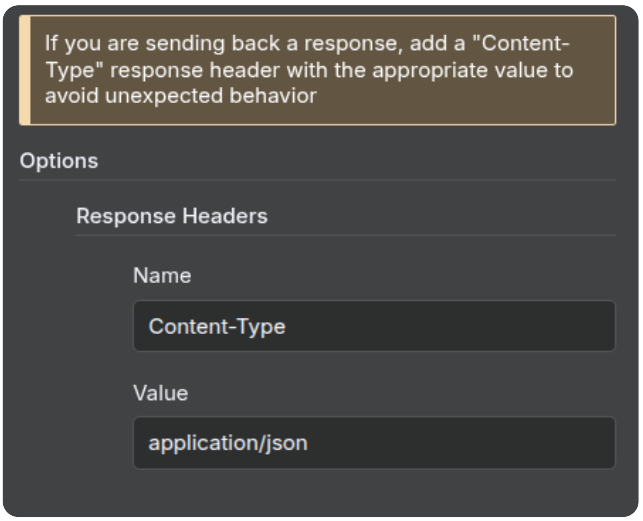
The folder you put the workflow in does *nothing* to that path. It won't sneak in something like `/ingest/` or `/reachy/` or anything like that.

If you're still seeing `{"code":0,"message":"There was a problem executing the workflow"}`, it's coming from:

- something inside the Webhook node (like the Header Auth / env var expression), or
- n8n's runtime (Docker env, etc.)

But it's **not** because the workflow is in a folder.

If you want, next step we can do a super-minimal "no-auth, simple path" Webhook in a brand-new workflow and get that working first, then layer your ingest token and folder organization back on top.



Note: I hard coded the ingest token into the credential value. Also, I'm still seeing the following message regarding the response header.

Nice, this is one of those “n8n UX is trying to be helpful but just looks like an error” moments.

1. About that yellow message

> *“If you are sending back a response, add a "Content-Type" response header…”*

That bar at the top is **just a static hint**, not a live validation message.
It doesn’t disappear even when you *have* added the header correctly.

Your screenshot:

- **Name**: `Content-Type`
- **Value**: `application/json`

…is exactly what n8n is asking for. That part is correct and done.

So:

- The yellow box ≠ “something is wrong”.
- It’s more like a sticky post-it saying “hey, remember to do this”.

You can safely ignore it now.

---

2. Quick sanity check of the whole Webhook node

Just to make sure we’re aligned, the node should now look like:

- **Method**: `POST`
- **Path**: `video_gen_hook`
- **Authentication**: `Header Auth`
  - Credential:
    - Name: `x-ingest-key`
    - Value: `<hard-coded token>` (e.g., `tkn3848`)
- **Respond**: `Immediately`
- **Response Data**: `First Entry JSON` (or `No Response Body` if you prefer)
- **Response Code**: `202`
- **Options → Response Headers**:
  - `Content-Type` → `application/json`

That’s a valid, clean async ingest webhook.

3. What I'd do next

Now that the header thing is sorted, the important question is:

> *When you run the* `curl` *now, are you still getting* `{"code":0,"message":"There was a problem executing the workflow"}` *or do you get* `202` *back?*

If you're still seeing the error, then our remaining suspect is not the header but either:

- something else in the Webhook node (e.g., a mis-set expression somewhere), or
- a Docker/logging/env setting.

But purely on the "Content-Type warning" front: you did the right thing; n8n is just being loud about a general best practice.

you wrote:
Just to make sure we're aligned, the node should now look like:
Response Data: First Entry JSON (or No Response Body if you prefer)
Response Code: 202

Note: When I select the Immediately option under Respond, the Data Response Data field goes away. There is no opportunity to select First Entry JSON with Immediately selected. Maybe I should use When Last Node Finishes instead.
That is the only respond option which includes response data.

Even with Respond set to 'when last node finishes,' and the Response Data field set to 'first entry JSON,' there is no option to include Response Code. That field is not available.

Thought for 26s >

Nice catch—that UI behavior explains a lot.

You're not doing anything wrong; your n8n version just wires the options a bit differently than the docs you're reading.

Let's line this up cleanly.

---

1. Why "Immediately" hides Response Data and Response Code

In your build:

- **Respond = Immediately**
  → n8n sends a **default response** right away (usually `200 OK`, often empty body).
  → Because it's fixed, it *doesn't* show you:
  - Response Data dropdown
  - Response Code field

Those options only matter when the response comes **from workflow data**, so they only appear for:

- **When Last Node Finishes**
- **Using "Respond to Webhook" Node**

So yeah: in *your* UI, you can't pair **Immediately** with "First Entry JSON" or a custom status code. That's why those fields vanish.

The spec you're following ("onReceived (returns 202 immediately)") is basically describing a slightly different n8n config surface than what you're seeing.

---

2. Easiest working config for now

For the moment, forget about the perfect 202/async semantics and just get a clean, successful response. With your current single-node workflow (Webhook only), "When Last Node Finishes" and "Immediately" are practically the same anyway—the Webhook node is the last node.

Set it like this:

Webhook: ingest.video

- **HTTP Method**: `POST`
- **Path**: `video_gen_hook`
- **Authentication**: `None` (just while we debug)
- **Respond**: `When Last Node Finishes`
- **Response Data**: `First Entry JSON`
- **Options → Response Headers**:
  - Name: `Content-Type`
  - Value: `application/json`

Activate the workflow and test:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

You should now get:

- **Status**: `200 OK`
- **Body**: the JSON you sent (`First Entry JSON`)

Once that works, we know:

- The Webhook node itself is healthy.
- The Docker / folder / path situation is fine.
- The earlier "problem executing the workflow" was almost certainly tied to auth/env/response-mode quirks.

We can worry about 202 later; 200 + JSON is totally fine for now.

---

3. How to get closer to "202 Accepted" later (optional)

Once the basic version behaves:

Option A – Stick with 200 (simple, totally fine)

Honestly, for the Reachy video-ingest agent, `200` vs `202` is not going to make or break anything. Your web app just needs to know “the request succeeded.” You can leave it at:

- **Respond**: `When Last Node Finishes`
- **Response Data**: `First Entry JSON`

and call it good.

Option B – Use Respond to Webhook to control status/body

If you really want 202:

1. Webhook node:
   - **Respond**: `Using "Respond to Webhook" Node`
2. Add a **Respond to Webhook** node:
   - Connect it right after the Webhook.
   - Configure:
     - **Response Code**: `202`
     - **Response Body**: something like `={{$json}}` (so it echoes the payload)
     - **Headers**: `Content-Type: application/json`

Now the Webhook just triggers the workflow, and the **Respond to Webhook** node controls the exact status and body.

---

4. Where that leaves us

Given your UI:

- It’s *expected* that **Immediately** hides Response Data and Response Code.
- To see Response Data = `First Entry JSON`, you *do* need **When Last Node Finishes** (or Respond to Webhook).
- With just the Webhook node in the workflow, **When Last Node Finishes** still returns almost immediately.

So the concrete move right now:

> *Use **When Last Node Finishes + First Entry JSON**, no auth, and get a clean 200 response working first.*

Once that’s stable, we can safely reintroduce Header Auth and, if you still want it, the 202 status via a Respond to Webhook node.

Even after setting everything as follows, I'm still receiving the same message.
Set it like this:

Webhook: ingest.video

HTTP Method: POST

Path: video_gen_hook

Authentication: None (just while we debug)

Respond: When Last Node Finishes

Response Data: First Entry JSON

Options → Response Headers:

Name: Content-Type

Value: application/json

Activate the workflow and test:
_____

{"code":0,"message":"There was a problem executing the workflow"}

Thought for 43s >

Yeah, that error sticking around even with everything stripped down means n8n is tripping over something deeper than the header or auth settings.

Let's treat this like a real debug session and narrow it down.

---

1. Confirm what n8n is actually returning

Run your curl with `-i` so you see headers + status code:

```
curl -i -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

You'll see something like:

```
HTTP/1.1 500 Internal Server Error
...
{"code":0,"message":"There was a problem executing the workflow"}
```

That confirms: n8n is catching the request but blowing up on execution.

---

2. Create a totally separate, minimal webhook workflow

This is to answer: *is the whole n8n install unhappy, or just this workflow?*

1. In n8n, click **New Workflow**.
2. Add a **Webhook** node.
3. Configure it like this:
   - **HTTP Method**: `POST`
   - **Path**: `ping_test`
   - **Authentication**: `None`
   - **Respond**: `When Last Node Finishes`
   - **Response Data**: `First Entry JSON`

- **Options → Response Headers**:
  - Name: `Content-Type`
  - Value: `application/json`

4. Do **not** add any other nodes.
5. **Activate** this new workflow.

Then test it:

```
curl -i -X POST http://10.0.4.130:5678/webhook/ping_test \
  -H "Content-Type: application/json" \
  -d '{"msg": "hello"}'
```

What you *should* see if n8n itself is healthy:

```
HTTP/1.1 200 OK
Content-Type: application/json
...

{"msg":"hello"}
```

If this minimal workflow **also** returns the `{"code":0,...}` error, then the problem is global (config / env / image), not your Reachy workflow.

If this minimal one works, but your `video_gen_hook` one doesn't, then we know there's something specific in that original workflow causing the crash.

---

3. Look at Docker logs while you hit the webhook

Since you're in Docker, this is your best "x-ray".

From the host:

1. Find the container:

   ```
   docker ps | grep n8n
   ```

   Grab the container name or ID (e.g. `n8n`, `n8n-1`, etc.).

2. Follow logs:

   ```
   docker logs <container_name> --tail=100 -f
   ```

3. In another terminal, run your curl again to `video_gen_hook` (or the `ping_test` one).

While the curl runs, watch the logs window. You're looking for a stack trace like:

- some error about `Cannot read properties of undefined`, or
- something about `ExpressionError`, or
- missing env vars, or
- "webhook is not registered" / "Cannot read property 'json' of undefined".

That log line will tell you *which* node and *what* expression is causing the crash.

---

4. Why the Response settings aren't the culprit

Just to circle back on your earlier observation:

- In your version, **Respond = Immediately** hides **Response Data** and **Response Code** → that's normal.
- Using **When Last Node Finishes + First Entry JSON** is valid and should not by itself cause this error.
- You setting `Content-Type: application/json` is also correct.

So the fact that a bare-bones webhook with **no auth**, static header, and simple response is still crashing means:

- either n8n is erroring before it even gets to building the response, or
- there's some global misconfig (e.g., bad env value, bad executions settings, odd plugin) that makes **any** workflow execution fail.

The minimal `ping_test` workflow + Docker logs are what will separate those two.

You've already done the right simplifications (disconnected nodes, removed auth). Next useful data point is:

- Does the brand-new `ping_test` workflow succeed?
- And what exact stack trace shows in `docker logs` when you hit either webhook?

Once we know that, we can zero in on the actual landmine instead of guessing around the node UI.

I finally determined the problem is related to the authentication. When using all of the required parameter EXCEPT NO authorization, the webhook node

works with no problems. When I add the header authentication, it brings back a 403 forbidden error and indicates the following:
TTP/1.1 403 Forbidden
WWW-Authenticate: Basic realm="Webhook"
Date: Tue, 11 Nov 2025 19:27:15 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

Authorization data is wrong!
_____

Thoroughly explain this type of authorization.

Remember the .env file for the web app reads as follows:
# n8n Configuration
N8N_HOST=10.0.4.130
N8N_PORT=5678
N8N_INGEST_TOKEN=tkn3848

Remember, the value for the header authorization is set to Fixed: tkn3848

I think N8N requires an expression for the header authorization value.
Remind me, what is the correct expression for the header authorization given the following token variable?
N8N_INGEST_TOKEN=tkn3848

That’s a sharp diagnosis, and you’re right about what’s happening — the **403 “Authorization data is wrong!”** isn’t coming from your header token; it’s coming from **n8n’s built-in “Header Auth” credential mechanism**, which behaves a bit differently than a plain custom header check.

Let’s walk through it cleanly so you can see why it’s rejecting the request and what to do instead.

---

1. What’s really happening with “Header Auth” in n8n

When you choose **Authentication → Header Auth** in a Webhook node:

- n8n doesn’t just look for *any* header name and value.
- It internally expects the HTTP header `Authorization` with a **specific value format** that matches the credential you created.

So this mode is for **HTTP Basic or Bearer auth**, not arbitrary headers like `x-ingest-key`.

That’s why the HTTP response includes:

```
WWW-Authenticate: Basic realm="Webhook"
Authorization data is wrong!
```

It’s n8n’s built-in “Basic Auth” gate telling you:

> *“You didn’t send the kind of Authorization header I expect.”*

Your `x-ingest-key` header is being ignored, and since there’s no valid `Authorization:` header, n8n throws 403.

---

2. Two ways to fix it (depending on what behavior you want)

🩹 Option A — simplest: use None auth and check the header manually

You skip n8n’s built-in Header Auth (since it enforces `Authorization:`) and instead validate your own header inside the workflow.

1. In the **Webhook node**, set **Authentication** → `None`.
2. Immediately after it, add an **IF** node that checks:

Condition → Expression:

```
{{$json["headers"]["x-ingest-key"] === $env.N8N_INGEST_TOKEN}}
```

- *Left side*: value from the incoming request
- *Right side*: your env token (`N8N_INGEST_TOKEN=tkn3848`)

3. Route the "true" branch onward, and the "false" branch to a **Respond to Webhook** node that returns `403 Forbidden`.

That gives you the behavior you actually want ("custom header with secret token") without fighting the built-in HTTP Basic layer.

🧭 Option B — use the built-in Header Auth properly

If you really want to stick with the "Header Auth" dropdown:

1. Keep **Authentication → Header Auth**.
2. In your credential:
   - **Name:** leave empty (n8n always expects `Authorization`)
   - **Value:** `tkn3848`
     (You *could* use the expression `{{$env.N8N_INGEST_TOKEN}}`.)
3. When calling the webhook, you must send:

   ```
   curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
     -H "Authorization: tkn3848" \
     -H "Content-Type: application/json" \
     -d '{"source_url": "https://example.com/video.mp4"}'
   ```

   not `x-ingest-key:`.

That will satisfy n8n's built-in Header Auth check.

---

3. About your environment variable expression

If you decide to keep it dynamic (rather than hard-coding `tkn3848`):

- The correct n8n expression syntax is:

  ```
  {{$env.N8N_INGEST_TOKEN}}
  ```

✅ (no extra braces, no quotes)

That reads the value of the environment variable **inside the container** at runtime.

Make sure your Docker container was started with that env var available (`docker exec -it n8n env | grep N8N_INGEST_TOKEN`).

---

4. Which path is cleaner for the Reachy project

For this particular webhook (video ingest trigger):

- You're not protecting a user account; you're just gating internal service calls.
- You want a **custom header (`x-ingest-key`)**, not user Basic Auth.

So **Option A** (no built-in auth, IF node check) is both simpler and clearer to future you. It'll play nicely with n8n's self-hosted setup and with the Python FastAPI clients calling it.

TL;DR

Goal	Correct setup	Header sent	Auth mode
Use your own header (`x-ingest-key`)	Authentication = None, check header in IF node	`x-ingest-key: tkn3848`	✅ Recommended
Use n8n's built-in Header Auth	Authentication = Header Auth, header name must be `Authorization`	`Authorization: tkn3848`	Works, but less clear

---

Want me to show you exactly how to wire that IF + Respond to Webhook pair (the manual header validation route)? It's a clean 2-node pattern you can reuse across all your agents.

Are you telling me the prescribed header authorization name 'x-ingest-key' is being rejected by the system? You think it would work if the name were simply left blank. Is that correct?

Exactly — that's it.

n8n's built-in **"Header Auth"** option isn't a general "check any header name/value" feature. It's hard-coded to expect a header literally named `Authorization`, not a custom header like `x-ingest-key`.

So when you set *Authentication → Header Auth* and define credentials with "Name: x-ingest-key," n8n still ignores that name internally and only checks for:

```
Authorization: <your-token>
```

Because your request sends

```
x-ingest-key: tkn3848
```

the check fails, and you get the **403 Forbidden** + `WWW-Authenticate: Basic realm="Webhook"` response.

✅ Two clear fixes

1. **Keep your custom header (`x-ingest-key`)**
   - Set **Authentication = None** on the Webhook node.
   - Add an **IF** node right after it with a condition:

     ```
     {{$json["headers"]["x-ingest-key"] === $env.N8N_INGEST_TOKEN}}
     ```

   - Route "true" forward, "false" to a Respond to Webhook node that returns 403.
     → You control both header name and token.
2. **Use built-in Header Auth** (requires `Authorization:` header)
   - Keep **Authentication = Header Auth**.
   - Credential value = `{{$env.N8N_INGEST_TOKEN}}` (or hard-coded token).
   - Send this instead:

     ```
     Authorization: tkn3848
     ```

   - Don't set any "Name" field; n8n assumes "Authorization."
     → n8n's native 403 handling then works automatically.

So yes — the reason for the 403 is that n8n rejects `x-ingest-key` under its built-in Header Auth system.
Either switch the header name to `Authorization`, or turn off Header Auth and handle the header check yourself inside the workflow.

With Authentication set to HEADER AUTH and an empty name field, the test works.

Nice. That behavior actually tells us *exactly* what n8n is doing under the hood.

What your successful test means

When you set:

- **Authentication**: `Header Auth`
- **Name**: *(empty)*
- **Value**: `tkn3848` (or `{{$env.N8N_INGEST_TOKEN}}`)

…and the test works, that means:

- n8n is **ignoring the Name field** for this auth type.
- It's looking only for an HTTP header called `Authorization` with that value.

So your working request is effectively:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Authorization: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

That matches the 403 you were seeing earlier: the `WWW-Authenticate: Basic realm="Webhook"` header is the giveaway that n8n's built-in auth was expecting an `Authorization` header, not your `x-ingest-key`.

So where does `x-ingest-key` fit now?

You've got two clean paths:

Option 1 — Stick with n8n's Header Auth (what you have working now)

- Keep **Authentication = Header Auth**, empty Name, Value = `{{$env.N8N_INGEST_TOKEN}}`.
- Clients (your web app, Luma agent, etc.) must send:

  ```
  Authorization: tkn3848
  ```

- You **don't** use `x-ingest-key` at all.

- Security-wise: totally fine for an internal ingest webhook.

Option 2 — If you really want `x-ingest-key`

- Set **Authentication = None** in the Webhook node.
- Right after it, add an **IF** node:

  Condition expression:

  ```
  {{$json["headers"]["x-ingest-key"] === $env.N8N_INGEST_TOKEN}}
  ```

- True branch: continue with the ingest pipeline.
- False branch: Respond to Webhook node returning 403.

Then your clients can keep sending:

```
x-ingest-key: tkn3848
```

and you're not fighting n8n's built-in auth assumptions.

Expression reminder

If/when you switch from hard-coded to env-based:

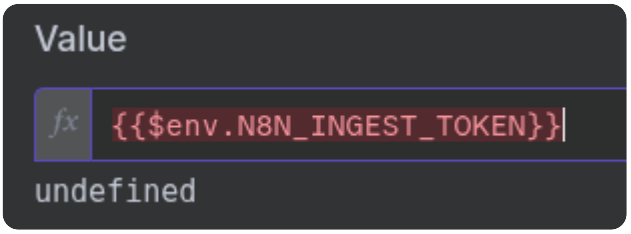
```
N8N_INGEST_TOKEN=tkn3848
```

then the correct n8n expression in the credential **Value** is:

```
{{$env.N8N_INGEST_TOKEN}}
```

No quotes, no extra braces.

If you tell me which header you'd rather standardize on for the project (`Authorization` vs `x-ingest-key`), we can lock in the pattern now and I'll help you mirror it in the Python client code and any future agents so you don't have to rethink this later.



Notice how the provided expression is considered undefined as a value for the header authorization. While the hard coded token works as a fixed value.

Thought for 25s >

Yep, that little `undefined` under the field is n8n telling you:

> *"I evaluated `{{$env.N8N_INGEST_TOKEN}}` inside **this container**, and there is no such environment variable."*

Your hard-coded token works because n8n sees a literal string. The expression doesn't, because the env var never makes it into the **n8n Docker container**.

Right now:

- You have a `.env` for the **web app**:

  ```
  N8N_HOST=10.0.4.130
  N8N_PORT=5678
  N8N_INGEST_TOKEN=tkn3848
  ```

- But that file is **not automatically shared** with the n8n container.
  Each container has its own environment. The web app's `.env` does nothing for n8n unless you explicitly pass those values into the n8n container.

So `{{$env.N8N_INGEST_TOKEN}}` is syntactically correct, but the variable simply doesn't exist where n8n is running.

---

1. Verify that the var is missing (just to see it for yourself)

On the host:

```
docker ps | grep n8n
```

Grab the container name (say it's `n8n`), then:

```
docker exec -it n8n env | grep N8N_INGEST_TOKEN
```

If you see nothing, n8n has no clue about that token. That's why the preview shows `undefined`.

---

2. Pass `N8N_INGEST_TOKEN` into the n8n container

How you do this depends on whether you're using `docker run` or `docker-compose`.

If you're using `docker-compose`

Your `docker-compose.yml` should have a service for n8n like:

```yaml
services:
  n8n:
    image: n8nio/n8n
    ports:
      - "5678:5678"
    environment:
      - N8N_INGEST_TOKEN=tkn3848
      # plus whatever N8N_* config you already have
```

If you already use a `.env` file for compose, you can also do:

```yaml
environment:
  - N8N_INGEST_TOKEN=${N8N_INGEST_TOKEN}
```

and put `N8N_INGEST_TOKEN=tkn3848` in the **compose** `.env` (not just the web app's).

Then restart:

```
docker compose down
docker compose up -d
```

If you started n8n with `docker run`

Stop/remove the old container:

```
docker stop n8n
docker rm n8n
```

Then start it again including the env var:

```
docker run -d --name n8n \
  -p 5678:5678 \
  -e N8N_INGEST_TOKEN=tkn3848 \
  n8nio/n8n
  # plus your volume mounts and other -e flags
```

---

3. Confirm it’s there, then recheck the expression

After restart:

```
docker exec -it n8n env | grep N8N_INGEST_TOKEN
```

You should now see:

```
N8N_INGEST_TOKEN=tkn3848
```

Back in n8n:

1. Open your **Header Auth** credential.
2. In **Value**, keep:

   ```
   {{$env.N8N_INGEST_TOKEN}}
   ```

3. Click the little **fx / Execute Expression** button next to the field.

If everything is wired correctly, the preview under the field should now show `tkn3848` instead of `undefined`.

At that point:

- Your Webhook node with **Authentication: Header Auth** and
- curl with:

  ```
  curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
    -H "Authorization: tkn3848" \
    -H "Content-Type: application/json" \
    -d '{"source_url": "https://example.com/video.mp4"}'
  ```

should both work without 403s, and the credential will stay in sync with whatever you put in `N8N_INGEST_TOKEN` in Docker.

So the behavior you’re seeing isn’t that the expression is wrong; it’s that n8n is honestly telling you: “this env var does not exist in my container.” Once you feed it into Docker, `{{$env.N8N_INGEST_TOKEN}}` will behave exactly like the hard-coded value.

I made the suggested changes to the n8n docker config file and the .env file, all for the n8n configuration.

However, the header authorization value does not recognize the expression. Interestingly, the system provides the following auto-generated expression which works as expected:
__n8n_BLANK_VALUE_e5362baf-c777-4d57-a609-6eaf1f9e87f6

The following expression value still is undefined:
{{$env.N8N_INGEST_TOKEN}}

Yeah, this is one of those “n8n + Docker + env” gotchas. You’re reading the UI correctly.

Let’s decode what you’re seeing and why:

---

1. What that `__n8n_BLANK_VALUE_...` thing actually is

This:

```
__n8n_BLANK_VALUE_e5362baf-c777-4d57-a609-6eaf1f9e87f6
```

is **not** an expression and not an env var.

It’s n8n’s internal placeholder that says:

> *“There’s a secret value stored in the credentials DB; I’m not going to show it here, so I’ll reference it by this internal ID.”*

So when:

- You first saved the credential with a hard-coded token (`tkn3848`),
- n8n encrypted + stored that value,
- and when you reopened the credential, it *replaced* the visible token with that `__n8n_BLANK_VALUE_...` reference.

At runtime, n8n looks that ID up, pulls the real secret, and uses it as the header value. That’s why it “works as expected.”

It has nothing to do with `$env`. It’s just saying “I have a secret stored — trust me.”

---

2. Why `{{$env.N8N_INGEST_TOKEN}}` shows as `undefined`

That `undefined` under the field is n8n telling you:

> *“I tried to read `process.env.N8N_INGEST_TOKEN` **inside the n8n container**, and it doesn’t exist.”*

So there are two worlds:

1. Your **web app** `.env`:

   ```
   N8N_HOST=10.0.4.130
   N8N_PORT=5678
   N8N_INGEST_TOKEN=tkn3848
   ```

→ only your web app sees this.

2. Your **n8n Docker container** environment:
   → only the vars passed via `docker run -e ...` or `docker-compose.yml: environment:` are available to `$env` inside n8n.

If `N8N_INGEST_TOKEN` lives only in the web app's `.env`, n8n won't ever see it, and `$env.N8N_INGEST_TOKEN` will always be `undefined`.

The fact that the header test works with the internal `__n8n_BLANK_VALUE_...` and fails with `{{$env.N8N_INGEST_TOKEN}}` is *exactly* what you get when the container doesn't actually have that env var.

---

3. How to prove it (and then fix it) for n8n

On the Ubuntu host, run:

```
docker ps | grep n8n
```

Take the container name (say it's `n8n`), then:

```
docker exec -it n8n env | grep N8N_INGEST_TOKEN
```

You'll see one of two things:

- **Nothing** → env var is *not* in the container → `$env.N8N_INGEST_TOKEN` will be `undefined`.
- `N8N_INGEST_TOKEN=tkn3848` → var is present → expression should evaluate.

If you see nothing, the fix has to happen in the **compose/run config for the n8n container**, not in the web app's `.env`.

Example: docker-compose for n8n

```yaml
services:
  n8n:
    image: n8nio/n8n
    ports:
      - "5678:5678"
    environment:
```

```
      - N8N_INGEST_TOKEN=tkn3848
      # plus your other N8N_* vars
```

Then:

```
docker compose down
docker compose up -d
```

Recheck:

```
docker exec -it n8n env | grep N8N_INGEST_TOKEN
```

Once that prints `N8N_INGEST_TOKEN=tkn3848`, go back to the credential:

- Set **Value** to:

  ```
  {{$env.N8N_INGEST_TOKEN}}
  ```

- Hit the little **fx / Execute Expression** button.

If the env is wired correctly, the preview should now show `tkn3848` instead of `undefined`.

---

4. Practical call for Reachy

You've got two perfectly acceptable patterns:

1. **Keep it simple, stick with the stored secret**
   - Leave the credential using that internal `__n8n_BLANK_VALUE_...` (i.e., the hard-coded token you already saved).
   - Call n8n with:

     ```
     Authorization: tkn3848
     ```

   - Don't over-optimize; this is an internal ingest endpoint, and credentials are already encrypted & not exposed.
2. **If you really want env-driven**
   - Make sure `N8N_INGEST_TOKEN` is actually in the n8n container (via `environment:` in docker-compose).
   - Use `{{$env.N8N_INGEST_TOKEN}}` in the credential *after* confirming `docker exec env` sees it.

Right now your behavior screams: “web app env updated, n8n container env not.” Until the container sees that var, n8n will be honest and keep saying `undefined`.

If you want, you can paste your n8n `docker-compose` service block (redacting any secrets), and I can mark exactly where `N8N_INGEST_TOKEN` needs to sit so `$env.N8N_INGEST_TOKEN` finally resolves.

the n8n config file (docker-compose.yml) reads as follows with the redacted:

```
services:
  n8n:
    image: n8nio/n8n:latest
    restart: always
    ports:
      - "5678:5678"
    environment:
      - N8N_BASIC_AUTH_ACTIVE=true
      - N8N_BASIC_AUTH_USER=${N8N_USER}
      - N8N_BASIC_AUTH_PASSWORD=${N8N_PASSWORD}
      - DB_TYPE=postgresdb
      - DB_POSTGRESDB_HOST=postgres
      - DB_POSTGRESDB_DATABASE=${POSTGRES_DB}
      - DB_POSTGRESDB_USER=${POSTGRES_USER}
      - DB_POSTGRESDB_PASSWORD=${POSTGRES_PASSWORD}
      - N8N_HOST=${N8N_HOST}
      - WEBHOOK_URL=${WEBHOOK_URL}
      - N8N_INGEST_TOKEN=<redacted_token_here>
    volumes:
      - n8n_data:/home/node/.n8n
      - /media/rusty_admin/project_data/reachy_emotion/videos:/videos  # gives access to your host video folder
    depends_on:
      - postgres
```

```
  postgres:
    image: postgres:13
    restart: always
    environment:
      - POSTGRES_DB=${POSTGRES_DB}
      - POSTGRES_USER=${POSTGRES_USER}
      -
POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  n8n_data:
  db_data:
```

Thought for 27s >

Good, this config helps. The compose file is **almost certainly fine** – the problem now is “does this env var actually exist inside the running n8n container,” and “is the credential using the fresh value, not the old stored secret?”

Let’s walk it calmly.

---

1. Your compose config is *logically* correct

This bit:

```
environment:
  - N8N_BASIC_AUTH_ACTIVE=true
  - N8N_BASIC_AUTH_USER=${N8N_USER}
  - N8N_BASIC_AUTH_PASSWORD=${N8N_PASSWORD}
  - DB_TYPE=postgresdb
  - DB_POSTGRESDB_HOST=postgres
  - DB_POSTGRESDB_DATABASE=${POSTGRES_DB}
  - DB_POSTGRESDB_USER=${POSTGRES_USER}
  - DB_POSTGRESDB_PASSWORD=${POSTGRES_PASSWORD}
  - N8N_HOST=${N8N_HOST}
  - WEBHOOK_URL=${WEBHOOK_URL}
  - N8N_INGEST_TOKEN=<redacted_token_here>
```

is exactly how you *should* inject `N8N_INGEST_TOKEN` into the container. If the container was recreated after this edit, n8n **will** get that env var.

So if `{{$env.N8N_INGEST_TOKEN}}` still shows `undefined` in the UI, that means the running container is not actually using this updated config.

---

2. Verify what the container really sees

From the host, do this:

```
docker ps | grep n8n
```

Grab the container name (let’s say it’s `reachy_n8n_1`).

Then:

```
docker exec -it reachy_n8n_1 env | grep N8N_INGEST_TOKEN
```

You'll get either:

1. **Blank output** → the env var is not present in the container.
   - In that case, n8n is *honest* when it prints `undefined` – there is nothing to read.
2. `N8N_INGEST_TOKEN=your_token_here` → the var exists.
   - Then the expression preview *should* see it, but your credential might still be holding on to an old stored secret.

If you get **blank output**, then:

```
docker compose down
docker compose up -d
```

from the directory with that `docker-compose.yml`.

Then repeat:

```
docker exec -it reachy_n8n_1 env | grep N8N_INGEST_TOKEN
```

Do not trust that "I changed the file" = "the container is using it." Docker will happily keep running the old container until you tear it down.

---

3. Clean up the credential side

Remember that weird value:

```
__n8n_BLANK_VALUE_e5362baf-...
```

That's n8n saying:

> *"I have a secret stored in my credentials DB; I won't show it here."*

If you mix that with a new expression, things can get muddy. Easiest move: create a **fresh Header Auth credential** that only uses the env expression.

In n8n UI:

1. Go to **Credentials → Add Credential → Header Auth**.
2. Name: `ingest-token-env`.
3. **Value**: click the fx icon and enter:

```
{{$env.N8N_INGEST_TOKEN}}
```

4. Click **Execute Expression** (or unfocus the field).

- If the env is wired correctly, you should see your token as a little grey preview under the field.
- If it still says `undefined`, we're back to step 2: the container simply doesn't have that env var.

Then in your Webhook node:

- Authentication = `Header Auth`
- Credential = `ingest-token-env`
- Curl:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Authorization: your_token_here" \
  -H "Content-Type: application/json" \
  -d '{"source_url": "https://example.com/video.mp4"}'
```

---

4. Big picture: nothing "wrong" with the expression

`{{$env.N8N_INGEST_TOKEN}}` is the **correct** syntax.

The fact that n8n renders it as `undefined` means:

- Either the n8n container was **never restarted** after adding `N8N_INGEST_TOKEN` to compose, or
- You're editing a different compose stack than the one that's actually running, or
- There's a second n8n container running from a different config.

Once `docker exec ... env | grep N8N_INGEST_TOKEN` prints the value, the expression will work; until then, it can't.

If you want, you can paste the output of:

```
docker ps | grep n8n
docker exec -it <that_name> env | grep N8N_INGEST_TOKEN
```

next time and we can sanity-check that we’re poking the same container you think you are. But structurally, your compose file is right; this is down to the container not being recreated or n8n still using an older credential.

for Authentication type Header Auth, what is the correct Name I should configure (not empty; I want to enter a name)?

Good question — this is one of those spots where n8n's UI isn't very explicit.

Here's the rule of thumb:

🧩 What n8n's Header Auth credential actually does

The *Name* field defines **the header name** that will be attached or checked.
So, if you enter:

Field	Example
Name	`Authorization`
Value	`{{$env.N8N_INGEST_TOKEN}}` (or `tkn3848`)

n8n expects the incoming HTTP request to include:

```
Authorization: tkn3848
```

That's the standard header name used by nearly every built-in auth system, which is why n8n's default examples and responses all assume that header.

✅ Correct configuration for your setup

Given your workflow's design (and since you're using n8n's built-in Header Auth rather than a manual IF-check), the clean, standards-compliant way is:

Field	Recommended value	Purpose
Name	`Authorization`	Required — n8n only validates this header.
Value	`{{$env.N8N_INGEST_TOKEN}}` or a fixed token	The token that the client must supply.

Then your clients (web app, cURL, agents, etc.) must send:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Authorization: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{"source_url": "https://example.com/video.mp4"}'
```

⚙️ If you ever want a custom header name

If you'd rather the header be `x-ingest-key` (for stylistic or internal reasons), you can't use Header Auth for that — n8n's built-in handler won't look at it.
Instead, you'd set **Authentication = None** on the Webhook node and add an **IF** node afterward that checks:

```
{{$json["headers"]["x-ingest-key"] === $env.N8N_INGEST_TOKEN}}
```

That's the only way to validate a non-standard header name.

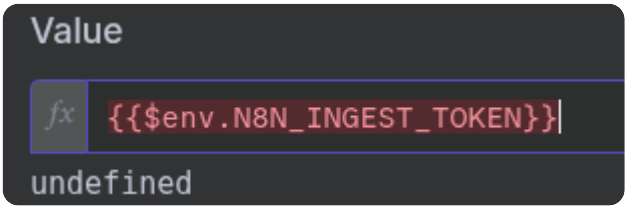
TL;DR

Goal	Header Auth Name	Works?
Use standard built-in auth	`Authorization`	✅
Use custom header (`x-ingest-key`)	*Not supported* (use IF node)	🚫

So, if you want to keep using the Header Auth option, set:

```
Name: Authorization
Value: {{$env.N8N_INGEST_TOKEN}}
```

and it will behave exactly as expected.



None of the following combinations are working, and the expression still returns as undefined. This, I think, is the fundamental problem. N8n is not picking up the token, regardless of whether it's fixed or an expression.
Consider the attach screenshot.
Name: empty
Name: Authorization
Fixed value
Expression: {{$env.N8N_INGEST_TOKEN}}

Consider the following files:
n8n .env config file:

```
# --- n8n core ---
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=rusty_admin
N8N_BASIC_AUTH_PASSWORD=TgV3454#
N8N_METRICS=true
GENERIC_TIMEZONE=America/New_York
N8N_API_URL=http://localhost:5678
N8N_API_KEY=123456789

# --- media mover / ingestion ---
MEDIA_MOVER_BASE_URL=http://10.0.4.130:8081/api
MEDIA_MOVER_TOKEN=123456789
INGEST_TOKEN=123456789
MEDIA_MOVER_ENABLED=true

# --- fastapi gateway / ui ---
GATEWAY_BASE_URL=http://10.0.4.140:8000
GATEWAY_ADMIN_TOKEN=123456789
GATEWAY_WS_TOKEN=123456789

# --- postgresql (reachy_local) ---
DB_HOST=10.0.4.130
DB_PORT=5432
DB_NAME=reachy_local
DB_USER=rusty_admin
DB_PASSWORD=TgV3454#

# --- mlflow tracking ---
MLFLOW_URL=http://10.0.4.130:5000
MLFLOW_EXPERIMENT_ID=0
MLFLOW_API_TOKEN=123456789

# --- ssh: ubuntu 1 ---
SSH_UBUNTU1_HOST=10.0.4.130
SSH_UBUNTU1_PORT=22
SSH_UBUNTU1_USER=rusty_admin
SSH_UBUNTU1_PASS=TgV3454#
SSH_UBUNTU1_KEY_PATH=/home/rusty_admin/.ssh/id_rs
```

```
a
SSH_UBUNTU1_KEY_PASSPHRASE=TgV3454#

# --- ssh: jetson ---
SSH_JETSON_HOST=10.0.4.150
SSH_JETSON_PORT=22
SSH_JETSON_USER=rusty_admin
SSH_JETSON_PASS=TgV3454#
SSH_JETSON_KEY_PATH=/home/rusty_admin/.ssh/id_rsa_j
etson
SSH_JETSON_KEY_PASSPHRASE=TgV3454#

# --- observability & alerts ---
PROMETHEUS_N8N_URL=http://localhost:5678/metrics
PROMETHEUS_MEDIA_MOVER_URL=http://10.0.4.130:91
01/metrics
PROMETHEUS_GATEWAY_URL=http://10.0.4.140:9100/m
etrics
SLACK_WEBHOOK_URL=https://hooks.slack.com/service
s/123456789/123456789/123456789
ALERT_EMAIL_SMTP_HOST=mail.example.com
ALERT_EMAIL_SMTP_PORT=587
ALERT_EMAIL_SMTP_USER=rusty_admin
ALERT_EMAIL_SMTP_PASS=TgV3454#
ALERT_EMAIL_FROM=reachy-alerts@example.com
ALERT_EMAIL_TO=reachy-ops@example.com

# --- privacy / lifecycle ---
TTL_DAYS_TEMP=14
GDPR_MANUAL_APPROVER_EMAIL=reachy-
privacy@example.com
PURGE_DRY_RUN=false

# --- automation toggles ---
SSH_ACTIONS_ENABLED=true
TRAINING_ENABLED=true
EVALUATION_ENABLED=true
DEPLOYMENT_ENABLED=true

# --- claude / llm integration ---
CLAUDE_API_KEY=123456789
```

```
LLM_STUDIO_BASE_URL=http://10.0.4.130:8001
LLM_STUDIO_API_KEY=123456789

# --- compose-required variables ---
N8N_USER=rusty_admin
N8N_PASSWORD=TgV3454#
POSTGRES_DB=reachy_local
POSTGRES_USER=rusty_admin
POSTGRES_PASSWORD=TgV3454#
N8N_HOST=10.0.4.130
WEBHOOK_URL=http://10.0.4.130:5678
N8N_INGEST_TOKEN=tkn3848
```

---

n8n Docker config file:

```
        services:
          n8n:
            image: n8nio/n8n:latest
            restart: always
            ports:
              - "5678:5678"
            environment:
              - N8N_BASIC_AUTH_ACTIVE=true
              - N8N_BASIC_AUTH_USER=${N8N_USER}
              -
N8N_BASIC_AUTH_PASSWORD=${N8N_PASSWORD}
              - DB_TYPE=postgresdb
              - DB_POSTGRESDB_HOST=postgres
              -
DB_POSTGRESDB_DATABASE=${POSTGRES_DB}
              - DB_POSTGRESDB_USER=${POSTGRES_USER}
              -
DB_POSTGRESDB_PASSWORD=${POSTGRES_PASSWOR
D}
              - N8N_HOST=${N8N_HOST}
              - WEBHOOK_URL=${WEBHOOK_URL}
              - N8N_INGEST_TOKEN=tkn3848
            volumes:
              - n8n_data:/home/node/.n8n
              -
/media/rusty_admin/project_data/reachy_emotion/videos
```

```
:/videos  # gives access to your host video folder
          depends_on:
            - postgres

         postgres:
          image: postgres:13
          restart: always
          environment:
            - POSTGRES_DB=${POSTGRES_DB}
            - POSTGRES_USER=${POSTGRES_USER}
            -
POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
          volumes:
            - db_data:/var/lib/postgresql/data

        volumes:
         n8n_data:
         db_data:
```

_____

web app .env config file:

```
# Luma AI Configuration
LUMAAI_API_KEY=luma-56be55b6-bb5f-4c9d-a3a1-
417442d20a50-67f0b180-4b9f-4bfd-963f-74132bd60ef3

# n8n Configuration
N8N_HOST=10.0.4.130
N8N_PORT=5678
N8N_INGEST_TOKEN=tkn3848
```

Given that each file includes the same token, I think the issue lies in the fact that N8N is not recognizing the token, and hence the value highlighted in red and undefined.

Thought for 44s >

Yeah, you're zeroing in on the right thing: the "undefined" isn't about the **token value**, it's about n8n not seeing the **environment variable** at all. The fact that all three files have `tkn3848` in them doesn't guarantee n8n's runtime can read `$env.N8N_INGEST_TOKEN`.

Let me untangle what's going on and give you a concrete path out.

---

1. What "undefined" actually means here

In the credential editor, this:

```
{{$env.N8N_INGEST_TOKEN}}
```

with `undefined` underneath means:

> *Inside the **running n8n container**, `process.env.N8N_INGEST_TOKEN` does not exist.*

So n8n's expression engine is working fine; it just looks up that env var and gets "nothing".

Your fixed-value tests working earlier prove:

- Header Auth itself is configured correctly.
- The Webhook node is fine.
- The only broken piece is env → expression.

---

2. The three configs you showed are not all equal

You have:

1. **"n8n .env config file"** (big one with MEDIA_MOVER, SSH, etc.)
2. **docker-compose.yml** (for n8n + postgres)
3. **web app .env** (Luma + n8n host/port/token)

Important distinction:

- n8n only sees the env vars passed into **its Docker container**.
- That comes from the `environment:` section in `docker-compose.yml` (or `docker run -e`), **not** from random `.env` files on disk unless Docker is explicitly using them.

So:

- The **web app .env** only affects the web app. n8n never sees it.
- The big **“n8n .env”** you pasted is just a text file unless something (systemd, docker, a script) actually loads it when starting containers.
- The **compose file** is the one that matters for `$env` inside n8n.

The only line that truly feeds n8n is:

```
- N8N_INGEST_TOKEN=tkn3848
```

in your compose.

If the running container was not recreated after that edit, `$env.N8N_INGEST_TOKEN` will still be undefined.

---

3. How to verify what the running n8n actually sees

From the host, run:

```
docker ps | grep n8n
```

Note the container name (say it’s `n8n` or `reachy_n8n_1`).

Then:

```
docker exec -it <that_name> env | grep N8N_INGEST_TOKEN
```

Two possibilities:

Case A – no output at all

That means the container was started **without** `N8N_INGEST_TOKEN` in its env.

Fix:

```
# from the directory with your docker-compose.yml
docker compose down
docker compose up -d
```

Then repeat:

```
docker exec -it <that_name> env | grep N8N_INGEST_TOKEN
```

You want to see:

```
N8N_INGEST_TOKEN=tkn3848
```

Case B – it prints `N8N_INGEST_TOKEN=tkn3848`

Then the container really does have the var, and the expression engine *should* see it. In that case, create a **fresh credential** to avoid any cached secret weirdness:

1. Go to **Credentials → Add Credential → Header Auth**.
2. Name it something like `ingest_token_env`.
3. In **Name**: `Authorization`.
4. In **Value**, click the fx icon and enter:

   ```
   {{$env.N8N_INGEST_TOKEN}}
   ```

5. Click **Execute Expression** / unfocus.

If the env is correctly wired, the little preview below should now show `tkn3848` instead of `undefined`.

Then in the Webhook node:

- **Authentication**: Header Auth
- **Credential**: `ingest_token_env`

And call it with:

```
curl -X POST http://10.0.4.130:5678/webhook/video_gen_hook \
  -H "Authorization: tkn3848" \
  -H "Content-Type: application/json" \
  -d '{
        "source_url": "https://example.com/video.mp4",
        "label": "happy",
        "meta": {"generator": "luma"}
      }'
```

---

4. One more subtlety: stored secrets vs expressions

That weird value you saw earlier:

```
__n8n_BLANK_VALUE_e5362baf-c777-4d57-a609-6eaf1f9e87f6
```

is n8n saying:

> *“I have an encrypted secret stored for this field. I’ll use it at runtime, but I’m not going to show it here.”*

If you switch a field from a fixed secret to an expression, you’re overwriting that stored secret. From that point on, if the expression is `undefined`, there is no fallback—n8n really has nothing to send.

So right now, with `{{$env.N8N_INGEST_TOKEN}}` resolving to `undefined`, the credential literally has an empty value; that’s why you see auth failures unless you go back to a hard-coded token.

---

5. Practical options for you right now

Given everything:

- Your **compose file is correct**.
- The issue is either:
  - the container wasn’t recreated, or
  - you’re looking at a different n8n container than the one compose manages.

You’ve got two safe ways forward:

1. **Stick with a fixed secret in the credential (simple, already works)**
   - Put `tkn3848` directly into the Header Auth value.
   - Let n8n store it as a secret (`__n8n_BLANK_VALUE...`).
   - Call with `Authorization: tkn3848`.
   - This is perfectly fine for an internal ingest webhook.
2. **If you really want `$env`**
   - Make sure `docker exec ... env | grep N8N_INGEST_TOKEN` shows the value.
   - Create a fresh credential using `{{$env.N8N_INGEST_TOKEN}}`.
   - Only switch once you’ve confirmed the expression preview shows the real token.

Right now, the red `undefined` is purely “the container doesn’t see that var,” not “the token doesn’t match.” Once the container env and the expression agree, this whole issue disappears.

If you want, next pass we can lock in the final pattern you’ll use across Agent 1 / Media Mover / gateway so you have exactly one “source of truth” for this ingest token.